OpenMP (Open Multi-Processing) Library

1. Introduction

OpenMP (Open Multi-Processing) is an application programming interface (API) that supports multi-platform shared-memory multiprocessing programming in C, C++, and Fortran. It allows programmers to easily write parallel code that runs across multiple CPU cores, dividing large tasks into smaller ones that can be executed simultaneously. The key goal of OpenMP is to improve performance and reduce execution time by utilizing the full power of modern processors.

2. Key Features of OpenMP

- Supports parallel programming on shared-memory systems.
- Simple to use with compiler directives (#pragma).
- Allows easy control over threads, loops, and data sharing.
- Portable and scalable across different hardware platforms.
- Incremental parallelization (you can convert sequential code gradually).

3. Example Code Using OpenMP

```
#include <stdio.h>
#include <omp.h>

int main() {
   int i;
   printf("Parallel region started:\n");

   #pragma omp parallel for
   for(i = 0; i < 8; i++) {
      printf("Thread %d is working on iteration %d\n", omp_get_thread_num(), i);
   }

   printf("Parallel region ended.\n");
   return 0;
}
```

4. Code Explanation

- #include <omp.h> → Includes the OpenMP header file.
- #pragma omp parallel for → Tells the compiler to run the for-loop in parallel using multiple threads.
- omp_get_thread_num() → Returns the ID of the thread currently executing.

- Each iteration of the loop is handled by a different thread to speed up processing.
- The number of threads is automatically chosen by the system (or can be set manually).

5. Expected Output

When you compile and run the program using:

```
gcc -fopenmp openmp_task.c -o openmp_task
./openmp_task
```

You may see output similar to:

```
Parallel region started:
Thread 0 is working on iteration 0
Thread 2 is working on iteration 2
Thread 1 is working on iteration 1
Thread 3 is working on iteration 3
Thread 0 is working on iteration 4
Thread 1 is working on iteration 5
Thread 2 is working on iteration 6
Thread 3 is working on iteration 7
Parallel region ended.
```

(The order may vary because threads run concurrently).

6. How It Works

- OpenMP divides the loop iterations among available CPU cores.
- Each core executes its part of the loop at the same time.
- The compiler and runtime system handle thread creation, scheduling, and synchronization.
- This parallelism reduces total computation time and improves performance on multi-core CPUs.